

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования «Тульский
государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине
Интеграция и тестирование программных систем
по теме
«Применение техник тест-дизайна»

Выполнил: ст. гр. 220032-11

Лаукерт К.М.

Принял: доцент

Басалов Ю.А.

Тула 2026

Лабораторная работа №3

по теме «Применение техник тест-дизайна»

Цель работы

Получить практические навыки применения техник тест-дизайна (эквивалентное разделение, анализ граничных значений, таблицы решений) для проектирования тестовых сценариев. Научиться формализовывать бизнес-правила в виде проверяемых тестовых условий и покрывать функциональность минимальным, но достаточным набором тестов; научиться писать простейшие unit-тесты на PyTest и выявлять дефекты в исходном коде модуля.

Описание тестируемой функции

Объект тестирования — функция `get_recommendations` (вариант 9, «Система рекомендации фильмов»): алгоритм подбора фильмов на основе предпочтений пользователя и истории просмотров.

Исходный код модуля взят из индивидуального задания и содержит преднамеренные ошибки, которые требуется выявить тестированием.

Параметр	Тип	Допустимые значения
age	int	0–120
favorite_genres	list	элементы из {comedy, drama, action, horror, sci-fi}
watched_recently	int	0–50 (за месяц)
rating_preference	float	0–10 (мин. рейтинг IMDb)
is_premium	bool	true / false

Бизнес-правила (ожидаемое поведение системы):

ID	Правило
R1	Если возраст < 12: исключить жанр «horror»
R2	Если возраст > 60: добавить 20% к приоритету жанра «drama»
R3	Если watched_recently > 20: снизить рекомендации на 30% (чтобы не надоедать)
R4	Если rating_preference > 8.5: показывать только фильмы с рейтингом >= 8.5 (фильтровать строго)
R5	Если is_premium: добавить 2 дополнительные рекомендации из категории «premium»
R6	Базовая логика: подобрать 10 фильмов по жанрам с учётом весов (по 3 фильма на жанр, рейтинги 7.5 / 7.8 / 8.1)

Исходный код тестируемого модуля (`recommendation_system.py`):

```
# -*- coding: utf-8 -*-
```

```
"""Вариант 9: Система рекомендации фильмов (ЛР3).
```

```
Алгоритм подбора фильмов на основе предпочтений пользователя и истории
```

```
просмотров. Исходный код модуля взят из индивидуального задания (репозиторий
```

```
преподавателя, lab3/option9.md) и содержит преднамеренные ошибки, которые
```

```
требуется выявить тестированием.
```

```
"""
```

```
def get_recommendations(age, favorite_genres, watched_recently, rating_preference, is_premium):
```

```
    recommendations = []
```

```
    genre_weights = {genre: 1.0 for genre in favorite_genres}
```

```
    if age < 12:
```

```
        if "horror" in genre_weights:
```

```
            del genre_weights["horror"]
```

```
    if age > 60:
```

```
        if "drama" in genre_weights:
```

```
            genre_weights["drama"] *= 1.2
```

```
    view_factor = 1.0
```

```
    if watched_recently > 20:
```

```
        view_factor = 0.7
```

```
    for genre, weight in genre_weights.items():
```

```
        for i in range(int(3 * weight * view_factor)):
```

```
            movie = {
```

```
                "title": f"{genre} movie {i}",
```

```
                "rating": 7.5 + (i * 0.3),
```

```
                "genre": genre,
```

```
                "is_premium": i % 3 == 0
```

```
    }

    if movie["rating"] >= rating_preference:
        recommendations.append(movie)

if is_premium:
    for i in range(2):
        recommendations.append({
            "title": f"Premium exclusive {i}",
            "rating": 9.0,
            "genre": "premium",
            "is_premium": True
        })

return recommendations[:10]
```

Эквивалентное разделение (EP)

Для каждого входного параметра выделены классы эквивалентности (валидные В и невалидные Н) и выбраны по одному репрезентативному значению. Для невалидных входных данных ожидаемое поведение — возврат сообщения об ошибке (значение вне диапазона не должно обрабатываться молча).

Параметр	Класс	Диапазон / описание	Представитель	Ожидаемое поведение
age	H1	возраст < 0	-1	сообщение об ошибке
age	B1	0–11 (ребёнок)	8	действует R1: horror исключается
age	B2	12–60 (взрослый)	30	возрастные правила не действуют
age	B3	61–120 (старший)	65	действует R2: +20% приоритета drama
age	H2	возраст > 120	121	сообщение об ошибке
favorite_genres	B1	непустой список валидных жанров	["comedy", "action"]	базовая генерация фильмов
favorite_genres	B2	содержит «horror»	["horror", "comedy"]	позволяет проверить R1
favorite_genres	B3	содержит «drama»	["drama"]	позволяет проверить R2
favorite_genres	H1	пустой список	[]	сообщение об ошибке
favorite_genres	H2	жанр вне допустимого набора	["documentary"]	сообщение об ошибке
watched_recently	H1	< 0	-1	сообщение об ошибке
watched_recently	B1	0–20	10	без снижения рекомендаций
watched_recently	B2	21–50	25	действует R3: снижение на 30%
watched_recently	H2	> 50	51	сообщение об ошибке
rating_preference	H1	< 0	-0.5	сообщение об ошибке
rating_preference	B1	0–8.5	7.0	без строгого фильтра (R4 не действует)
rating_preference	B2	8.6–10	9.0	действует R4: строгий фильтр >= 8.5
rating_preference	H2	> 10	10.5	сообщение об ошибке
is_premium	B1	true	True	действует R5: +2 рекомендации premium
is_premium	B2	false	False	без премиум-рекомендаций

Примечание: граница 8.5 принадлежит классу B1, так как правило R4 сформулировано строго («если rating_preference > 8.5»); значение 8.6 — минимальное класса B2. Параметр is_premium — булев, классов эквивалентности ровно два.

Анализ граничных значений (BVA)

Для каждого параметра с упорядоченным диапазоном определены границы диапазонов и границы срабатывания бизнес-правил; проверяются значения на границе, за границей и внутри диапазона:

Параметр	Граница	Проверяемые значения	Ожидаемое поведение
age	диапазон 0–120	0 и 120 (границы); 30 (внутри)	валидны, базовое поведение
age	правило R1 (< 12)	11 и 12	при 11 horror исключается; при 12 — нет
age	правило R2 (> 60)	60 и 61	при 60 без надбавки; при 61 — +20% drama (4 фильма)
age	за диапазоном	-1 и 121	сообщение об ошибке
watched_recently	диапазон 0–50	0 и 50 (границы); 10 (внутри)	валидны
watched_recently	правило R3 (> 20)	20 и 21	при 20 — 3 фильма; при 21 — 2 (снижение на 30%)
watched_recently	за диапазоном	-1 и 51	сообщение об ошибке
rating_preference	диапазон 0–10	0.0 и 10.0 (границы); 7.0 (внутри)	валидны
rating_preference	правило R4 (> 8.5)	8.5 и 8.6	при 8.5 фильтра нет — 3 фильма; при 8.6 — строгий фильтр (0 фильмов)
rating_preference	за диапазоном	-0.5 и 10.5	сообщение об ошибке
favorite_genres	неупорядоченный параметр — граничные случаи по длине списка	0 (пустой), 1, 4	0 — ошибка; 1 и 4 — валидны

Таблица решений

Логика функции зависит от комбинации пяти условий: C1 (возраст < 12), C2 (возраст > 60), C3 (watched_recently > 20), C4 (rating_preference > 8.5), C5 (is_premium). Полная таблица содержит 3 (класс возраста) × 2 × 2 × 2 = 24 комбинации; по методике немотивированные комбинации объединены — условия, не влияющие на исход проверяемого правила, зафиксированы в нейтральном значении. Каждый столбец-правило преобразован в тестовый сценарий:

Правило	Возраст	C3: watched > 20	C4: pref > 8.5	C5: premium	Действия системы	Тест-кейсы
П1	12–60	нет	нет	нет	базовые 3 фильма на жанр, лимит 10	TC-REC-01
П2	0–11	нет	нет	нет	R1: horror исключается	TC-REC-02, 03
П3	0–11	да	нет	нет	R1 + R3	TC-REC-16
П4	61–120	нет	нет	нет	R2: +20% приоритета drama	TC-REC-04, 05
П5	61–120	да	нет	нет	R2 + R3	TC-REC-17
П6	12–60	да	нет	нет	R3: снижение на 30%	TC-REC-06
П7	12–60	нет	да	нет	R4: строгий фильтр >= 8.5	TC-REC-08, 09
П8	12–60	нет	нет	да	R5: +2 рекомендации premium	TC-REC-13
П9	12–60	нет	нет	да (>= 10 базовых фильмов)	R5 + R6: 10 базовых + 2 premium	TC-REC-14
П10	12–60	нет	нет	нет (4 жанра)	R6: усечение до 10 фильмов	TC-REC-15

Итоговая таблица тест-кейсов

Объединены результаты всех техник (EP, BVA, таблица решений), дублирующие сценарии удалены. Формат входных данных: age; favorite_genres; watched_recently; rating_preference; is_premium.

ID	Название	Входные данные	Ожидаемый результат
TC-REC-01	Базовый сценарий (взрослый, 2 жанра)	30; [comedy, action]; 10; 7.0; False	6 фильмов (по 3 на жанр), жанры — только избранные
TC-REC-02	Ребёнок: исключение horror (R1)	8; [horror, comedy]; 10; 7.0; False	horror отсутствует; comedy — 3 фильма
TC-REC-03	BVA: граница правила R1 по возрасту	11 и 12; [horror, comedy]; 10; 7.0; False	при 11 horror исключён; при 12 — присутствует

TC-REC-04	Старший: +20% приоритета drama (R2)	65; [drama]; 10; 7.0; False	4 фильма drama ($3 \times 1.2 = 3.6 \rightarrow 4$)
TC-REC-05	BVA: граница правила R2 по возрасту	60 и 61; [drama]; 10; 7.0; False	при 60 — 3 фильма; при 61 — 4
TC-REC-06	BVA: граница правила R3	30; [comedy]; 20 и 21; 7.0; False	при 20 — 3 фильма; при 21 — 2 (снижение на 30%)
TC-REC-07	BVA: границы диапазона 0–50	30; [comedy]; 0 и 50; 7.0; False	при 0 — 3 фильма; при 50 — 2
TC-REC-08	Строгий фильтр рейтинга (R4)	30; [comedy]; 10; 9.0; False	0 фильмов (в базе нет фильмов с рейтингом ≥ 8.5)
TC-REC-09	BVA: граница правила R4	30; [comedy]; 10; 8.5 и 8.6; False	при 8.5 фильтра нет — 3 фильма; при 8.6 — 0
TC-REC-10	R4 не должен действовать при $\text{pref} \leq 8.5$	30; [comedy, action]; 10; 8.0; False	6 фильмов (все фильмы избранных жанров)
TC-REC-11	Нижняя граница $\text{rating_preference} = 0$	30; [comedy, action]; 10; 0.0; False	6 фильмов
TC-REC-12	Верхняя граница $\text{rating_preference} = 10$	30; [comedy]; 10; 10.0; False	0 фильмов (строгий фильтр)
TC-REC-13	Premium: +2 рекомендации (R5)	30; [comedy, action]; 10; 7.0; True	8 фильмов, из них 2 — категории premium
TC-REC-14	R5 + R6: премиум не отсекается лимитом 10	30; [comedy, action, drama, sci-fi]; 10; 7.0; True	10 базовых + 2 premium
TC-REC-15	R6: базовый лимит 10 фильмов	30; [comedy, action, drama, sci-fi]; 10; 7.0; False	ровно 10 фильмов
TC-REC-16	Комбинация R1 + R3 (таблица решений, П3)	8; [horror, comedy]; 25; 7.0; False	только comedy, 2 фильма ($3 \times 0.7 = 2.1 \rightarrow 2$)
TC-REC-17	Комбинация R2 + R3 (таблица решений, П5)	65; [drama]; 25; 7.0; False	2 фильма drama ($3 \times 1.2 \times 0.7 = 2.52 \rightarrow 2$)
TC-REC-18	Негативный: возраст < 0	-1; [comedy]; 10; 7.0; False	сообщение об ошибке
TC-REC-19	Негативный: возраст > 120	121; [comedy]; 10; 7.0; False	сообщение об ошибке
TC-REC-20	Негативный: $\text{watched_recently} < 0$	30; [comedy]; -1; 7.0; False	сообщение об ошибке
TC-REC-21	Негативный: $\text{watched_recently} > 50$	30; [comedy]; 51; 7.0; False	сообщение об ошибке
TC-REC-22	Негативный: $\text{rating_preference} < 0$	30; [comedy]; 10; -0.5; False	сообщение об ошибке
TC-REC-23	Негативный: $\text{rating_preference} > 10$	30; [comedy]; 10; 10.5; False	сообщение об ошибке
TC-REC-24	Негативный: несуществующий жанр	30; [documentary]; 10; 7.0; False	сообщение об ошибке
TC-REC-25	Негативный: пустой список жанров	30; []; 10; 7.0; False	сообщение об ошибке

ТС- REC-26	BVA: границы диапазона возраста 0–120	0 и 120; [comedy]; 10; 7.0; False	3 фильма (значения валидны)
---------------	--	--------------------------------------	--------------------------------

Исходный код тестов на языке Python

Тесты реализованы библиотекой PyTest (Python 3.13, pytest 9.1.1). Каждый тест-кейс из итоговой таблицы закодирован как отдельная тестовая функция; сценарии с наборами граничных значений (11/12, 60/61, 20/21, 0/50, 8.5/8.6, 0/120) и все негативные сценарии реализованы через параметризацию `@pytest.mark.parametrize` с человекочитаемыми идентификаторами (ids). Тесты фиксируют ожидаемое поведение по спецификации: падение теста означает найденный дефект исходного кода.

```
# -*- coding: utf-8 -*-
```

```
"""ЛР3, вариант 9: тесты функции get_recommendations («Система  
рекомендации фильмов»).
```

Тест-кейсы TC-REC-01..26 спроектированы по техникам ЕР, BVA и
таблица решений

(см. отчёт). Тесты фиксируют ожидаемое поведение ПО СПЕЦИФИКАЦИИ
задания;

падение теста означает найденный дефект исходного кода (DEF-
01..04 в отчёте).

Базовая генерация фильмов: по 3 фильма на жанр (i = 0, 1, 2),
рейтинги 7.5 / 7.8 / 8.1.

```
"""
```

```
import pytest
```

```
from recommendation_system import get_recommendations
```

```
def genres_of(result):  
    return {m["genre"] for m in result}
```

```
def count_genre(result, genre):  
    return sum(1 for m in result if m["genre"] == genre)
```

```

# TC-REC-01: базовый сценарий – взрослый, 2 жанра, без
смягчающих правил
def test_base_case():
    result = get_recommendations(30, ["comedy", "action"], 10,
7.0, False)
    assert isinstance(result, list)
    assert len(result) == 6
    assert genres_of(result) == {"comedy", "action"}

# TC-REC-02: R1 – возраст < 12 исключает жанр horror
def test_child_horror_excluded():
    result = get_recommendations(8, ["horror", "comedy"], 10,
7.0, False)
    assert "horror" not in genres_of(result)
    assert count_genre(result, "comedy") == 3

# TC-REC-03: BVA правила R1 – граница возраста 11/12
@pytest.mark.parametrize("age,horror_expected", [(11, False),
(12, True)],
                        ids=["age-11", "age-12"])
def test_horror_age_boundary(age, horror_expected):
    result = get_recommendations(age, ["horror", "comedy"], 10,
7.0, False)
    assert ("horror" in genres_of(result)) is horror_expected

# TC-REC-04: R2 – возраст > 60 даёт +20% приоритета drama (3 *
1.2 = 3.6 -> 4)
def test_senior_drama_boost():
    result = get_recommendations(65, ["drama"], 10, 7.0, False)
    assert len(result) == 4

# TC-REC-05: BVA правила R2 – граница возраста 60/61
@pytest.mark.parametrize("age,expected_count", [(60, 3), (61,
4)],

```

```

        ids=["age-60", "age-61"])
def test_drama_boost_boundary(age, expected_count):
    result = get_recommendations(age, ["drama"], 10, 7.0, False)
    assert len(result) == expected_count

# TC-REC-06: BVA правила R3 – граница watched_recently 20/21
(снижение на 30%)
@pytest.mark.parametrize("watched,expected_count", [(20, 3),
(21, 2)]),
        ids=["watched-20", "watched-21"])
def test_viewed_reduction_boundary(watched, expected_count):
    result = get_recommendations(30, ["comedy"], watched, 7.0,
False)
    assert len(result) == expected_count

# TC-REC-07: BVA диапазона watched_recently 0-50 – обе границы
валидны
@pytest.mark.parametrize("watched,expected_count", [(0, 3), (50,
2)]),
        ids=["watched-0", "watched-50"])
def test_viewed_range_bounds(watched, expected_count):
    result = get_recommendations(30, ["comedy"], watched, 7.0,
False)
    assert len(result) == expected_count

# TC-REC-08: R4 – rating_preference > 8.5: строгий фильтр (в
базе нет фильмов >= 8.5)
def test_strict_filter_high_pref():
    result = get_recommendations(30, ["comedy"], 10, 9.0, False)
    assert result == []

# TC-REC-09: BVA правила R4 – граница rating_preference 8.5/8.6
@pytest.mark.parametrize("pref,expected_count", [(8.5, 3), (8.6,
0)]),

```

```

        ids=["pref-8.5", "pref-8.6"])
def test_rating_filter_boundary(pref, expected_count):
    result = get_recommendations(30, ["comedy"], 10, pref,
False)
    assert len(result) == expected_count

# TC-REC-10: R4 – при pref <= 8.5 строгого фильтра быть не
должно
def test_moderate_pref_no_strict_filter():
    result = get_recommendations(30, ["comedy", "action"], 10,
8.0, False)
    assert len(result) == 6

# TC-REC-11: нижняя граница rating_preference = 0 (без фильтра)
def test_pref_zero_all_movies():
    result = get_recommendations(30, ["comedy", "action"], 10,
0.0, False)
    assert len(result) == 6

# TC-REC-12: верхняя граница rating_preference = 10 (строгий
фильтр)
def test_pref_upper_bound():
    result = get_recommendations(30, ["comedy"], 10, 10.0,
False)
    assert result == []

# TC-REC-13: R5 – is_premium добавляет 2 рекомендации premium (6
+ 2 = 8)
def test_premium_adds_two():
    result = get_recommendations(30, ["comedy", "action"], 10,
7.0, True)
    assert len(result) == 8
    assert count_genre(result, "premium") == 2

```

```

# TC-REC-14: R5 + R6 — премиум ДОПОЛНИТЕЛЬНЫЕ, не отсекаются
лимитом 10
def test_premium_not_cut_by_cap():
    result = get_recommendations(30, ["comedy", "action",
                                      "drama", "sci-fi"],
                                10, 7.0, True)
    assert count_genre(result, "premium") == 2

# TC-REC-15: R6 — базовый лимит: не более 10 фильмов
def test_base_cap_ten():
    result = get_recommendations(30, ["comedy", "action",
                                      "drama", "sci-fi"],
                                10, 7.0, False)
    assert len(result) == 10

# TC-REC-16: комбинация R1 + R3 (ребёнок + много просмотренного)
def test_child_heavy_watcher():
    result = get_recommendations(8, ["horror", "comedy"], 25,
    7.0, False)
    assert "horror" not in genres_of(result)
    assert len(result) == 2 # comedy: 3 * 0.7 = 2.1 -> 2

# TC-REC-17: комбинация R2 + R3 (старший + много просмотренного)
def test_senior_heavy_watcher():
    result = get_recommendations(65, ["drama"], 25, 7.0, False)
    assert len(result) == 2 # 3 * 1.2 * 0.7 = 2.52 -> 2

# TC-REC-18..25: негативные сценарии — невалидные входные данные
отвергаются
@pytest.mark.parametrize("args", [
    (-1, ["comedy"], 10, 7.0, False), # TC-REC-18: age < 0

```

```

    (121, ["comedy"], 10, 7.0, False),      # TC-REC-19: age >
120
    (30, ["comedy"], -1, 7.0, False),      # TC-REC-20:
watched_recently < 0
    (30, ["comedy"], 51, 7.0, False),      # TC-REC-21:
watched_recently > 50
    (30, ["comedy"], 10, -0.5, False),     # TC-REC-22:
rating_preference < 0
    (30, ["comedy"], 10, 10.5, False),     # TC-REC-23:
rating_preference > 10
    (30, ["documentary"], 10, 7.0, False), # TC-REC-24:
несуществующий жанр
    (30, [], 10, 7.0, False),              # TC-REC-25: пустой
список жанров
], ids=["age<0", "age>120", "watched<0", "watched>50",
        "pref<0", "pref>10", "invalid-genre", "empty-genres"])
def test_invalid_input_rejected(args):
    result = get_recommendations(*args)
    assert isinstance(result, str) # сообщение об ошибке, а не
список

# TC-REC-26: BVA диапазона возраста 0-120 – валидные крайние
значения
@pytest.mark.parametrize("age", [0, 120], ids=["age-0", "age-
120"])
def test_age_range_bounds(age):
    result = get_recommendations(age, ["comedy"], 10, 7.0,
False)
    assert len(result) == 3

```

Отчёт PyTest

Запуск: `py -m pytest -v` (из папки ЛР3; конфигурация `pytest.ini`: `pythonpath = ., testpaths = tests`). Итог: собрано 32 прогона (26 тест-кейсов, часть — параметризована) — 19 passed, 13 failed; каждое падение отнесено к одному из четырёх дефектов (см. следующий раздел).

```
===== test session starts
=====

platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 --
C:\Users\PC\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir:
C:\Users\PC\Documents\ТулГУ\6_семестр\Интеграция_и_тестирование\
ЛР3
configfile: pytest.ini
testpaths: tests
collecting ... collected 32 items

tests/test_recommendations.py::test_base_case PASSED
[ 3%]
tests/test_recommendations.py::test_child_horror_excluded PASSED
[ 6%]
tests/test_recommendations.py::test_horror_age_boundary[age-11]
PASSED [ 9%]
tests/test_recommendations.py::test_horror_age_boundary[age-12]
PASSED [ 12%]
tests/test_recommendations.py::test_senior_drama_boost FAILED
[ 15%]
tests/test_recommendations.py::test_drama_boost_boundary[age-60]
PASSED [ 18%]
tests/test_recommendations.py::test_drama_boost_boundary[age-61]
FAILED [ 21%]
tests/test_recommendations.py::test_viewed_reduction_boundary[wa
tched-20] PASSED [ 25%]
tests/test_recommendations.py::test_viewed_reduction_boundary[wa
tched-21] PASSED [ 28%]
tests/test_recommendations.py::test_viewed_range_bounds[watched-
0] PASSED [ 31%]
tests/test_recommendations.py::test_viewed_range_bounds[watched-
50] PASSED [ 34%]
```

```

tests/test_recommendations.py::test_strict_filter_high_pref
PASSED [ 37%]
tests/test_recommendations.py::test_rating_filter_boundary[pref-
8.5] FAILED [ 40%]
tests/test_recommendations.py::test_rating_filter_boundary[pref-
8.6] PASSED [ 43%]
tests/test_recommendations.py::test_moderate_pref_no_strict_filt
er FAILED [ 46%]
tests/test_recommendations.py::test_pref_zero_all_movies PASSED
[ 50%]
tests/test_recommendations.py::test_pref_upper_bound PASSED
[ 53%]
tests/test_recommendations.py::test_premium_adds_two PASSED
[ 56%]
tests/test_recommendations.py::test_premium_not_cut_by_cap
FAILED [ 59%]
tests/test_recommendations.py::test_base_cap_ten PASSED
[ 62%]
tests/test_recommendations.py::test_child_heavy_watcher PASSED
[ 65%]
tests/test_recommendations.py::test_senior_heavy_watcher PASSED
[ 68%]
tests/test_recommendations.py::test_invalid_input_rejected[age<0
] FAILED [ 71%]
tests/test_recommendations.py::test_invalid_input_rejected[age>1
20] FAILED [ 75%]
tests/test_recommendations.py::test_invalid_input_rejected[watch
ed<0] FAILED [ 78%]
tests/test_recommendations.py::test_invalid_input_rejected[watch
ed>50] FAILED [ 81%]
tests/test_recommendations.py::test_invalid_input_rejected[pref<
0] FAILED [ 84%]
tests/test_recommendations.py::test_invalid_input_rejected[pref>
10] FAILED [ 87%]
tests/test_recommendations.py::test_invalid_input_rejected[invalid-genre] FAILED [ 90%]
tests/test_recommendations.py::test_invalid_input_rejected[empty
-genres] FAILED [ 93%]
tests/test_recommendations.py::test_age_range_bounds[age=0]
PASSED [ 96%]
tests/test_recommendations.py::test_age_range_bounds[age=120]
PASSED [100%]

```

```

===== short test summary info
=====

FAILED tests/test_recommendations.py::test_senior_drama_boost -
AssertionErro...

FAILED
tests/test_recommendations.py::test_drama_boost_boundary[age=61]
- Ass...

FAILED
tests/test_recommendations.py::test_rating_filter_boundary[pref=
8.5]

FAILED
tests/test_recommendations.py::test_moderate_pref_no_strict_filt
er - A...

FAILED
tests/test_recommendations.py::test_premium_not_cut_by_cap -
Assertion...

FAILED
tests/test_recommendations.py::test_invalid_input_rejected[age<0
] - As...

FAILED
tests/test_recommendations.py::test_invalid_input_rejected[age>1
20] - ...

FAILED
tests/test_recommendations.py::test_invalid_input_rejected[watch
ed<0]

FAILED
tests/test_recommendations.py::test_invalid_input_rejected[watch
ed>50]

FAILED
tests/test_recommendations.py::test_invalid_input_rejected[pref<
0] - A...

FAILED
tests/test_recommendations.py::test_invalid_input_rejected[pref>
10] - ...

FAILED
tests/test_recommendations.py::test_invalid_input_rejected[invalid-genre]

FAILED
tests/test_recommendations.py::test_invalid_input_rejected[empty
-genres]

===== 13 failed, 19 passed in 0.08s
=====

```

Список ошибок в исходном коде модуля

По результатам прогона тестов в исходном коде выявлены четыре дефекта:

ID	Дефект	Описание	Выявившие тесты
DEF-01	Отсутствует валидация входных данных	Спецификация задаёт диапазоны всех параметров, однако функция не проверяет их: значения <code>age = -1/121</code> , <code>watched_recently = -1/51</code> , <code>rating_preference = -0.5/10.5</code> , несуществующий жанр «documentary» и пустой список жанров обрабатываются молча (например, отрицательный возраст воспринимается как «ребёнок» и исключает <code>horror</code> , невалидный жанр попадает в выдачу). Нарушено: таблица «Допустимые значения» спецификации. Ожидаемое поведение — возврат сообщения об ошибке.	TC-REC-18...25 (8 падений)
DEF-02	Фильтр по рейтингу применяется всегда	Код фильтрует фильмы условием <code>movie["rating"] >= rating_preference</code> при любом значении параметра, тогда как правило R4 предписывает применять строгий фильтр (<code>>= 8.5</code>) только при <code>rating_preference > 8.5</code> . Следствие: при <code>rating_preference <= 8.5</code> фильмы с рейтингом ниже порога ошибочно теряются — при <code>pref = 8.5</code> вместо 3 фильмов возвращается 0, при <code>pref = 8.0</code> вместо 6 возвращается 2. Правильная реализация: порог 8.5 и только при <code>pref > 8.5</code> .	TC-REC-09 (8.5), TC-REC-10
DEF-03	Премиум-рекомендации отсекаются лимитом	Премиум-фильмы добавляются в общий список до усечения <code>recommendations[:10]</code> , поэтому при 10 и более базовых фильмах оба премиум-фильма полностью отбрасываются — подписка <code>is_premium</code> не даёт эффекта. Нарушено правило R5 («добавить 2 дополнительные рекомендации»): дополнения должны добавляться сверх лимита либо учитываться в нём отдельно.	TC-REC-14
DEF-04	Надбавка приоритета drama не проявляется	Правило R2 (+20% к приоритету «drama») реализовано как <code>weight *= 1.2</code> , но из-за усечения <code>int()</code> прирост теряется: <code>int(3 × 1.2) = int(3.6) = 3</code> фильма — столько же, сколько без надбавки. Правило фактически не работает: при <code>age > 60</code> ожидается 4 фильма drama, возвращается 3.	TC-REC-04, TC-REC-05 (61)

Выводы

Спроектирован и выполнен набор из 26 тест-кейсов (32 прогона PyTest): 19 прошли, 13 упали — все падения сгруппированы в четыре дефекта исходного кода (DEF-01...DEF-04). Применённые техники тест-дизайна распределили ответственность за покрытие: эквивалентное разделение обеспечило проверку всех классов входных данных, включая негативные (именно они выявили полное отсутствие валидации — DEF-01, самый массовый дефект); анализ граничных значений оказался наиболее результативным — границы правил R1–R4 (11/12, 60/61, 20/21, 8.5/8.6) вскрыли три функциональных дефекта (DEF-02, DEF-04), а границы диапазонов подтвердили корректную обработку крайних валидных значений; таблица решений систематизировала комбинации правил — именно сочетание `is_premium` с лимитом R6 (правило П9) выявило отсечение премиум-рекомендаций (DEF-03). Полнота набора подтверждается: каждый класс эквивалентности каждого параметра покрыт хотя бы одним тестом, каждая граница каждого диапазона и каждого правила проверена с двух сторон, все столбцы таблицы решений преобразованы в сценарии, негативные сценарии охватывают все невалидные классы. Набор является минимально достаточным: дубликаты между техниками объединены (например, BVA-границы 11/12 одновременно закрывают класс EP «ребёнок»). Практический вывод: граничные значения и комбинации правил — наиболее дефектоёмкие места данной функции, поэтому при её доработке в первую очередь следует добавить валидацию входных данных, применять фильтр рейтинга только при `pref > 8.5` (порог 8.5), добавлять премиум-рекомендации сверх лимита 10 и реализовать надбавку приоритета `drama` так, чтобы она проявлялась в выдаче.